

Техническое	Задание	(ТЗ)
на разработку "Единой Цифровой Образовательной Платформы Университета" (ЕЦОПУ)		

Введение

- **1. Назначение документа:** Настоящее Техническое Задание (ТЗ) определяет требования к разработке и функциональности "Единой Цифровой Образовательной Платформы Университета" (далее – Платформа).
- **2. Наименование системы:** Полное: "Единая Цифровая Образовательная Платформа Университета". Краткое: ЕЦОПУ.

2. Цели и задачи проекта

- **2.1. Цели создания платформы:**
 - Создание централизованной цифровой экосистемы формата "единого окна" для управления междисциплинарными проектами, организации мероприятий (хакатоны, конференции, акселераторы), формирования студенческих команд и обмена знаниями.
 - Автоматизация и оптимизация процессов для заведующих кафедрами, преподавателей и студентов.
 - Повышение вовлеченности студентов и качества проектной деятельности.
 - Укрепление связей с индустриальными и государственными партнерами.
 - Обеспечение синергии между подразделениями (кафедрами, лабораториями), преподавателями и студентами.
- **2.2. Задачи, решаемые платформой:**
 - Автоматизация отчетности и аналитики.
 - Объединение студентов, преподавателей и партнеров в едином информационном пространстве.
 - Предоставление инструментов для управления полным жизненным циклом проектов.

- Облегчение организации и проведения образовательных и соревновательных мероприятий.
- Поддержка студенческих инициатив и их интеграция в образовательный процесс.
- Формирование и ведение цифровых портфолио всех участников.
- Автоматическое создание документов по стандартизированным шаблонам.

Раздел 3. Термины, определения и сокращения

Настоящий раздел определяет основные термины, определения и сокращения, используемые в данном Техническом Задании (ТЗ) и сопутствующей проектной документации. Целью данного раздела является обеспечение единого и однозначного понимания предметной области и функциональности платформы всеми участниками проекта.

3.1. Общие термины и определения

- **Платформа (Система)** — Программно-аппаратный комплекс "Единая Цифровая Образовательная Платформа Университета" (ЕЦОПУ), являющийся объектом разработки в рамках настоящего ТЗ. Представляет собой веб-приложение, доступное через браузер.
- **Единое окно** — Принцип организации интерфейса Платформы, при котором пользователь получает доступ ко всем необходимым функциям, данным и сервисам из одной точки входа, без необходимости переключаться между разными несвязанными системами.
- **Модуль** — Логически и функционально обособленная часть Платформы, отвечающая за конкретный набор бизнес-задач (например, "Проектный менеджмент", "Рейтинговая система"). Модули могут разрабатываться и обновляться относительно независимо друг от друга.
- **Дашборд (Информационная панель)** — Персонализированная стартовая страница пользователя после авторизации, агрегирующая в виде виджетов ключевую информацию, относящуюся к его роли и активностям (например, текущие проекты, ближайшие события, уведомления, ключевые KPI).
- **Виджет** — Информационный блок на Дашборде, отображающий конкретный срез данных в наглядном виде (например, виджет "Мои задачи", виджет "Календарь мероприятий").
- **Портфолио (Цифровое портфолио)** — Персональный, динамически обновляемый профиль пользователя, который аккумулирует и систематизирует все его достижения, подтвержденные навыки, опыт участия

в проектах и мероприятиях, полученные оценки, отзывы и награды в рамках Платформы.

- **Уведомление (Оповещение)** — Короткое информационное сообщение, доставляемое пользователю внутри Платформы (внутриплатформенное), по электронной почте (email) или через push-уведомления для информирования о важных событиях (например, новая задача, приближение дедлайна, приглашение в команду).
- **Шаблон** — Заранее подготовленный и настраиваемый образец для создания однотипных сущностей (например, шаблон проекта, шаблон отчета, шаблон сертификата), который упрощает и стандартизирует процессы.
- **Плейсхолдер** — Специальная метка в шаблоне документа (например, [ФИО_студента], [Название_проекта]), которая при генерации документа автоматически заменяется на реальные данные из системы.

3.2. Роли пользователей

3.3. Ключевые сущности Платформы

3.4. Технические термины и сокращения

Раздел 4. Требования и модули

Настоящий раздел определяет перечень функций, которые должны быть реализованы в "Единой Цифровой Образовательной Платформе Университета" (ЕЦОПУ). Функционал сгруппирован по системным модулям. Общая структура проекта показана на рисунке 1.

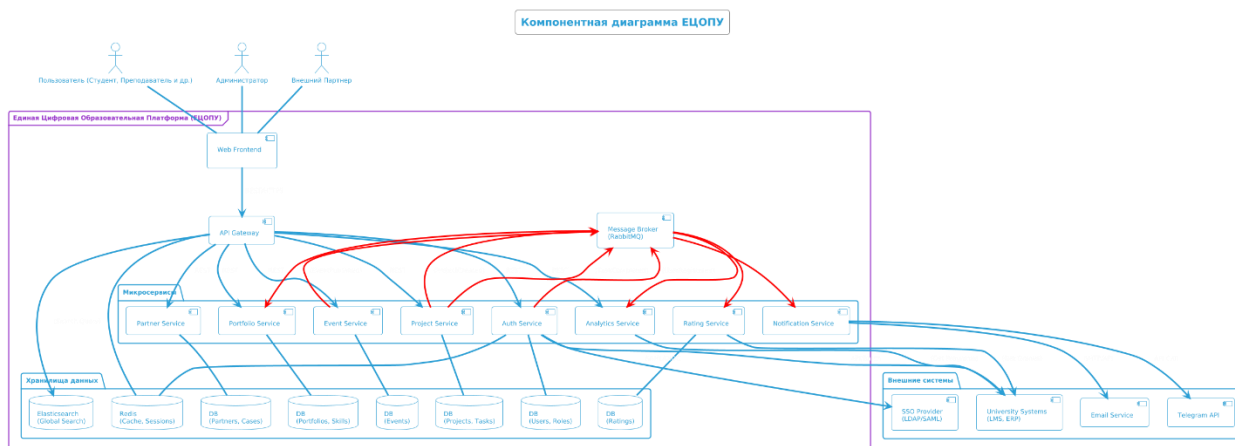


Рисунок 1 – Диаграмма компонентов планируемой платформы

4.1.1. Модуль 0: Ядро платформы и общие функции

Обеспечивает базовую инфраструктуру и сквозную функциональность для работы всех остальных модулей.

- **Подсистема аутентификации и авторизации:**

- Должна быть реализована функция регистрации новых пользователей с использованием корпоративной почты домена. Процесс регистрации должен включать отправку письма со ссылкой для верификации на указанный адрес.

- Должна быть обеспечена интеграция с университетской службой каталогов (LDAP/Active Directory) для реализации механизма единого входа (SSO) для ролей "Студент" и "Преподаватель".

- Для роли "Внешний партнер" должна быть предусмотрена отдельная процедура регистрации по email и паролю.

- Система должна предоставлять функцию восстановления пароля через электронную почту.

- Система должна реализовывать ролевую модель доступа (RBAC), строго разграничивающую права пользователей в соответствии с присвоенной им ролью.

- **Подсистема персонализированных информационных панелей (Дашбордов):**

- После авторизации каждый пользователь должен перенаправляться на персонализированный дашборд. Состав и содержание виджетов на дашборде должны определяться ролью пользователя:

- **Дашборд Студента:** Должен отображать виджеты: список активных проектов и задач с указанием сроков, список рекомендованных мероприятий, ленту уведомлений, индикатор текущего рейтинга.

- **Дашборд Преподавателя:** Должен отображать виджеты: список курируемых проектов, список студенческих заявок, требующих рассмотрения, календарь с запланированными мероприятиями.

- **Дашборд Зав. кафедрой:** Должен отображать виджеты с агрегированными KPI по кафедре: общее число проектов, процент вовлеченности студентов, динамика рейтинга кафедры.
 - Должна быть реализована возможность базовой кастомизации дашборда пользователем (скрытие, изменение порядка виджетов).
- **Подсистема уведомлений и оповещений:**
 - Система должна поддерживать три канала доставки уведомлений: внутриплатформенные (в интерфейсе), по электронной почте, push-уведомления (при наличии PWA/мобильного приложения).
 - Уведомления должны автоматически генерироваться при наступлении системных событий (назначение задачи, приглашение в проект, изменение статуса заявки, приближение дедлайна и т.д.).
 - В настройках профиля пользователь должен иметь возможность гибко сконфигурировать типы и каналы получаемых уведомлений.
- **Подсистема глобального поиска:**
 - Система должна быть оснащена единой поисковой строкой, доступной на всех ключевых страницах.
 - Поиск должен осуществляться по всем ключевым сущностям Платформы: проекты, мероприятия, пользователи (студенты, преподаватели), навыки, документы.
 - Страница результатов поиска должна содержать инструменты для расширенной фильтрации и сортировки найденных результатов (например, фильтр проектов по статусу, по кафедре; фильтр мероприятий по дате).
- **Подсистема коммуникаций:**
 - Для каждого проекта должен автоматически создаваться изолированный групповой чат для участников этого проекта. Чат должен поддерживать обмен текстовыми сообщениями и файлами.
 - Система должна обеспечивать возможность отправки личных сообщений между любыми двумя пользователями Платформы.

4.1.2. Модуль "Проектный менеджмент"

Обеспечивает инструментарий для полного жизненного цикла управления проектами. Общая схема показана на рисунке 2.

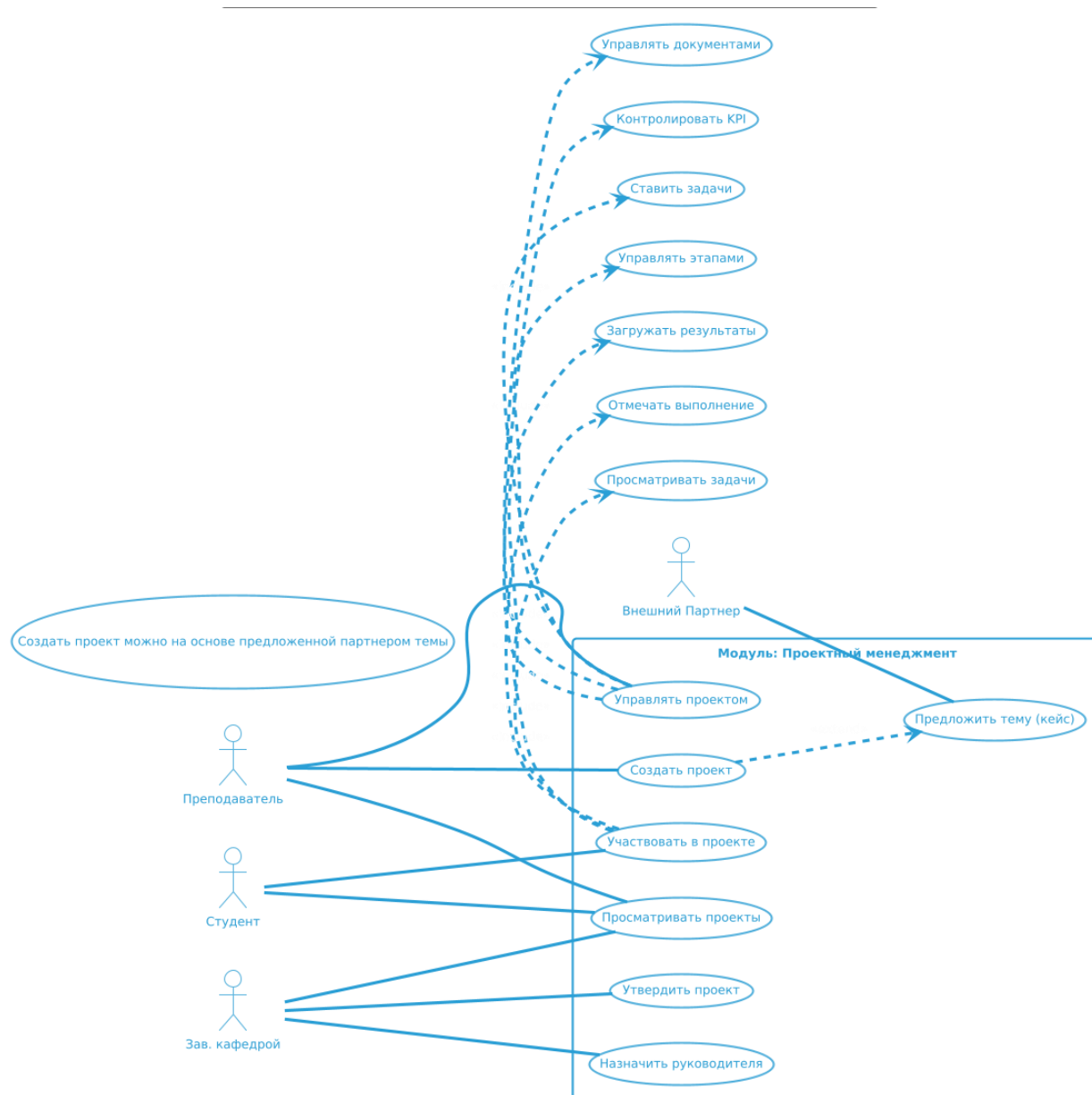


Рисунок 2 – Use case диаграмма модуля проектный менеджмент

- **Функции для Руководителей/Инициаторов:**

- Реализовать возможность создания проекта на основе предустановленных шаблонов (например, "Дипломный проект", "Научное исследование", "Коммерческий кейс") или в свободной форме.
- Предоставить интерфейс для определения атрибутов проекта: цели, задачи, KPI, сроки, необходимые ресурсы.

- Реализовать функционал декомпозиции проекта на этапы и задачи.
- Предоставить интерфейс для назначения задач конкретным участникам с указанием сроков выполнения.
- Реализовать инструмент для контроля выполнения задач, например, в виде Kanban-доски со статусами ("К выполнению", "В работе", "На проверке", "Выполнено").
- Обеспечить возможность загрузки и хранения проектной документации (ТЗ, отчеты, презентации) с поддержкой версионирования.

- **Функции для Участников:**

- Обеспечить просмотр назначенных задач и их деталей.
- Реализовать возможность изменения статуса задачи.
- Предоставить функцию загрузки результатов работы (файлов) к соответствующей задаче.

4.1.3. Модуль "Мероприятия (Хакатоны и акселераторы)"

Предоставляет инструменты для организации и участия в образовательных и соревновательных мероприятиях.

Примерная структура показана на рисунках 3 и 4.

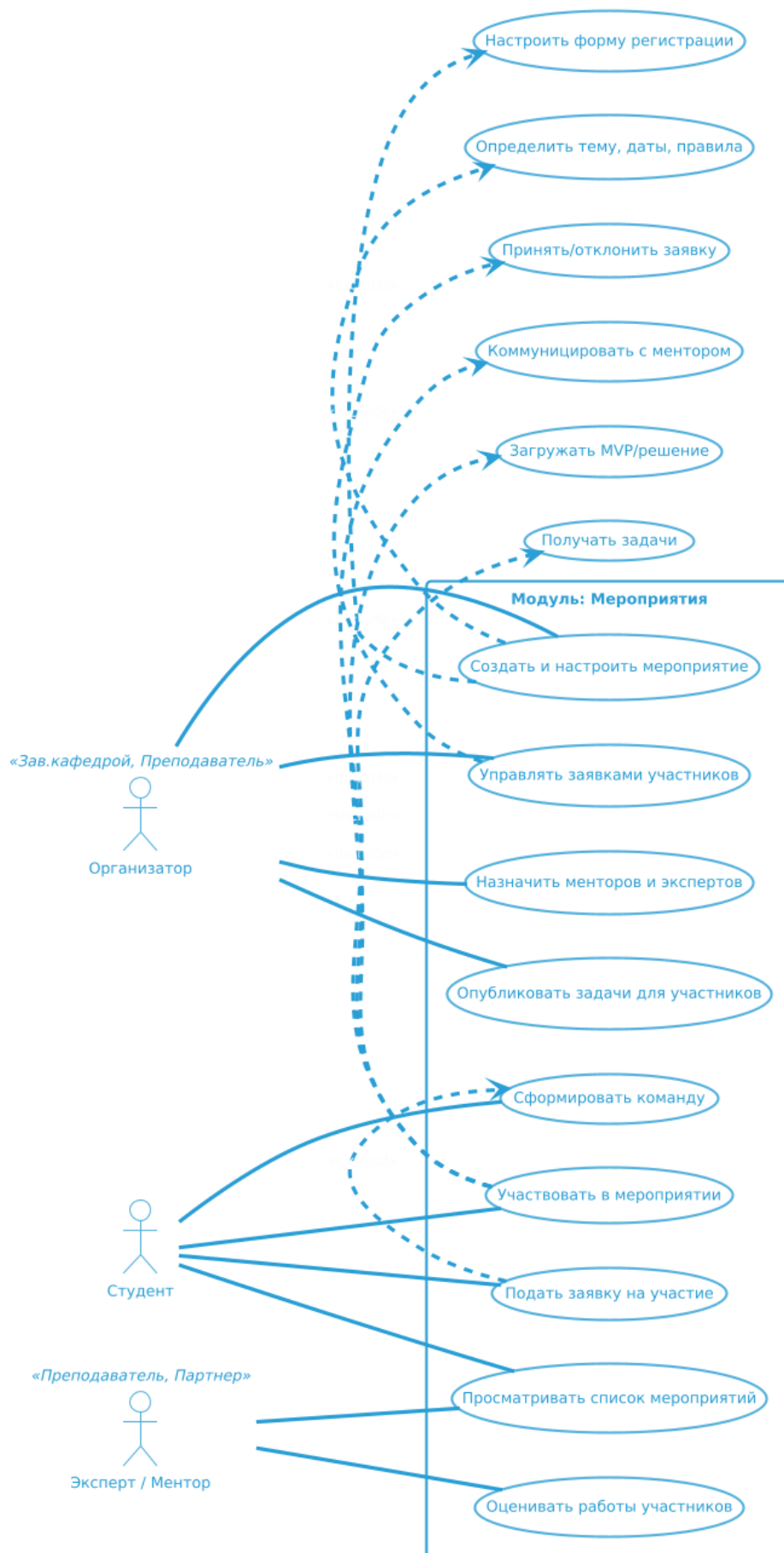


Рисунок 3 – Use Case диаграмма модуля мероприятий

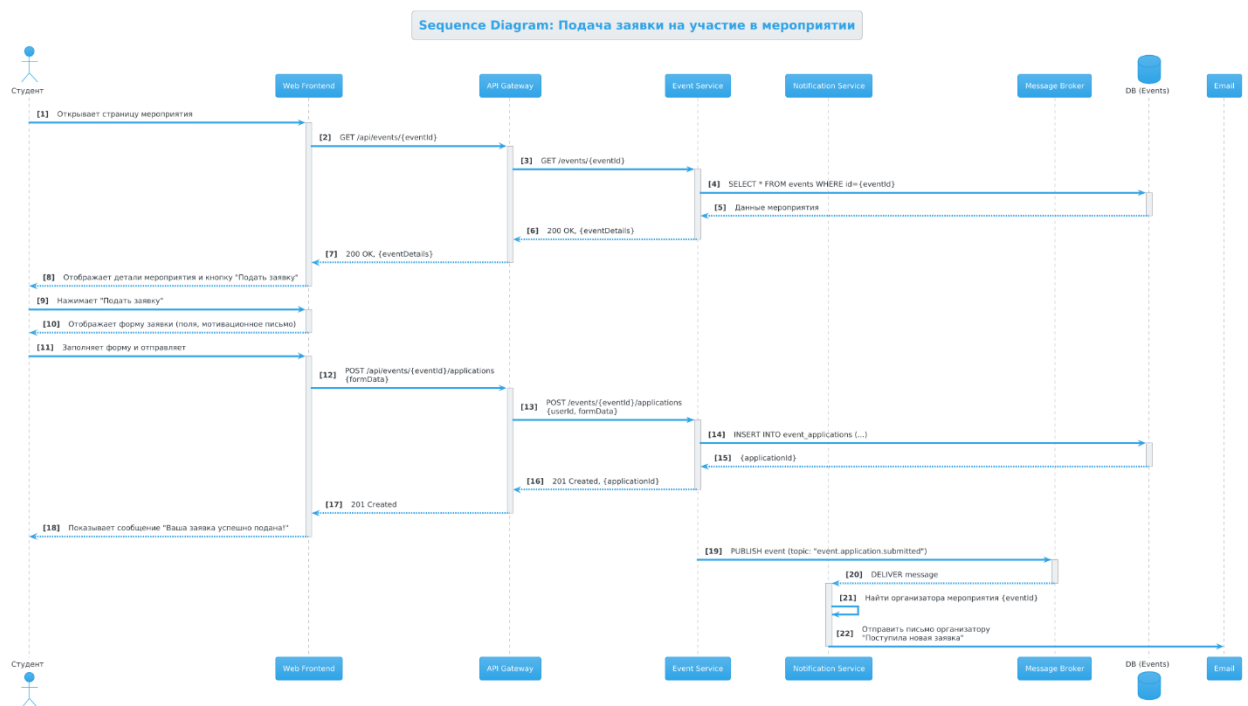


Рисунок 4 – Диаграмма последовательности регистрации студента на мероприятия

- **Функции для Организаторов:**

- Реализовать интерфейс для создания страницы мероприятия с указанием его атрибутов: тема, даты, формат (онлайн/офлайн/гибридный), правила, бюджет, критерии оценки.
- Предоставить конструктор форм регистрации для сбора заявок от участников.
- Обеспечить инструменты для управления заявками: просмотр, отбор, формирование команд (в ручном или автоматическом режиме).
- Реализовать возможность назначения Менторов и Экспертов для мероприятия.

- **Функции для Участников:**

- Предоставить доступ к общему списку мероприятий с возможностью фильтрации.
- Реализовать процесс подачи заявки на участие (индивидуально или в составе команды).
- Обеспечить интерфейс для взаимодействия с организаторами и менторами в ходе мероприятия (получение задач, загрузка решений/MVP).

4.1.4. Модуль "Цифровое портфолио"

Предназначен для автоматического формирования и ведения цифрового следа достижений пользователей.

- Система должна автоматически агрегировать и отображать в профиле пользователя следующие данные из других модулей:
 - Список проектов и мероприятий, в которых пользователь принимал участие, с указанием его роли.
 - Полученные сертификаты, дипломы и награды.
 - Подтвержденные навыки и компетенции.
 - Отзывы от руководителей проектов и менторов.

Должна быть реализована возможность ручного добавления пользователем внешних достижений (например, публикации, победы во внешних конкурсах) с возможностью прикрепления подтверждающих документов.

Должна быть реализована функция поиска пользователей по навыкам, опыту, интересам для формирования команд.

Доступ к просмотру портфолио другого пользователя должен предоставляться только с его явного согласия (например, при отклике на вакансию от партнера).

4.1.5. Модуль "Аналитика и отчетность"

Данный модуль предназначен для автоматизированного сбора, обработки и визуализации данных о деятельности на Платформе, а также для формирования стандартизированных и кастомных отчетов для руководящего состава.

- **Подсистема автоматической генерации отчетов:**

- Система должна обеспечивать формирование отчетов по следующим срезам с возможностью выгрузки в форматах .xlsx, .pdf, .csv:

- **Отчет по проектам:** Должен включать поля: название проекта, руководитель, кафедра, тип проекта, статус (активен, завершен, заморожен), даты начала и окончания, количество участников, процент

выполнения KPI, оценка успешности. Должна быть предусмотрена фильтрация по периоду, кафедре, руководителю, статусу.

- **Отчет по активности кафедр:** Должен содержать сводные данные по каждой кафедре за выбранный период: общее количество инициированных проектов, количество активных проектов, суммарное количество студентов-участников, процент вовлеченности студентов кафедры в проектную деятельность, средний рейтинг преподавателей и студентов кафедры, количество проведенных мероприятий.

- **Отчет по участникам (студентам и преподавателям):** Должен предоставлять индивидуальную статистику по каждому пользователю за период: количество проектов, в которых принял участие, роли в проектах, количество инициированных проектов (для преподавателей), количество выигранных конкурсов/грантов, средняя оценка за проекты, количество публикаций (если применимо).

- **Отчет по мероприятиям (событиям):** Должен включать данные: название мероприятия, организатор, даты проведения, количество поданных заявок, количество участников, количество сформированных команд, список победителей (если применимо).

- Должна быть реализована функция автоматической рассылки регулярных отчетов (например, ежемесячный отчет по активности кафедр для Зав. кафедрой) по заданному расписанию.

- **Подсистема отслеживания KPI:**

- Система должна в реальном времени отслеживать и отображать на соответствующих дашбордах ключевые показатели эффективности:

- **Для проектов:** Степень выполнения (процент завершенных задач), соблюдение сроков, освоение ресурсов (если применимо).

- **Для кафедр:** Уровень вовлеченности студентов, количество проектов на одного преподавателя, количество публикаций по результатам проектов.

- **Для Платформы в целом:** Общее количество пользователей, количество активных проектов, динамика создания новых проектов/мероприятий.

- Должна быть предусмотрена возможность добавления Администратором кастомных KPI для отслеживания.

- **Подсистема анализа компетенций:**

- Система должна агрегировать данные о навыках, указанных в портфолио пользователей и требуемых в проектах.

- Должна быть реализована функция сопоставления имеющихся у студентов компетенций с требованиями рынка труда (на основе импортируемых данных или интеграции с внешними сервисами, например, hh.ru API).

- Должна быть возможность визуализации "карты компетенций" по факультету или кафедре, показывающей наиболее развитые и недостающие навыки.

4.1.6. Модуль "Рейтинговая система"

Модуль предназначен для объективной и прозрачной оценки активности и успешности участников и подразделений с целью мотивации и здоровой конкуренции.

- **Расчет индивидуального рейтинга студента:**

Рейтинг должен рассчитываться как взвешенная сумма баллов, начисляемых за различные виды деятельности.

Критерии для начисления баллов (базовый список):

- Участие в проекте (базовые баллы).
- Успешное завершение проекта (дополнительные баллы).
- Роль в проекте (руководитель команды получает больше баллов, чем участник).
- Участие в мероприятии (хакатоне, конференции).
- Победа или призовое место в мероприятии (значительные баллы).
- Получение гранта.

- Публикация научной статьи по результатам проекта.
- Положительные отзывы от руководителя/ментора.
- Успеваемость (данные импортируются из LMS, если реализована интеграция).

Алгоритм расчета должен быть прозрачен для пользователя (в профиле должна быть детализация, за что и сколько баллов начислено).

Расчет индивидуального рейтинга преподавателя. Критерии для начисления баллов (базовый список):

- Количество инициированных и руководимых проектов.
- Успешность проектов под его руководством (процент успешно завершенных).
- Победы его подопечных в конкурсах и хакатонах.
- Количество публикаций по результатам проектов.
- Организация мероприятий.
- Выступление в роли ментора/эксперта.

Расчет рейтинга кафедр:

- Рейтинг должен формироваться на основе агрегированных показателей студентов и преподавателей, приписанных к данной кафедре.
- Критерии должны включать: суммарную активность, количество и качество проектов, количество организованных мероприятий, победы студентов на внешних конкурсах.

• **Управление рейтинговой системой:**

- Администратор Платформы должен иметь интерфейс для настройки весовых коэффициентов для каждого критерия, влияющего на рейтинг. Это позволит адаптировать систему мотивации под текущие стратегические задачи университета.
- Рейтинги должны отображаться в виде упорядоченных списков (топ-100 студентов, рейтинг кафедр) с возможностью визуализации в виде графиков и диаграмм.

4.1.7. Модуль "Портал для внешних партнеров"

Модуль предназначен для выстраивания эффективного взаимодействия университета с индустриальными и государственными партнерами.

- **Функции для партнеров:**

- Реализовать отдельный портал (раздел сайта) с возможностью регистрации представителей компаний.
- Предоставить интерфейс для публикации "кейсов" – практических задач, которые могут стать основой для студенческих проектов или тем для хакатонов.
- Обеспечить возможность публикации предложений по стажировкам и вакансиям, доступных студентам Платформы.
- Реализовать инструменты для прямого взаимодействия с заинтересовавшимися студенческими командами и преподавателями (встроенный мессенджер, система заявок).
- Предоставить доступ к публичным частям портфолио студентов (с их согласия) для подбора кадров.

- **Управление соглашениями:**

- Реализовать механизм для электронного обмена и подписания (или фиксации факта подписания) соглашений о сотрудничестве и NDA (соглашений о неразглашении) между партнером и командой проекта.

4.1.8. Модуль "Управление пользователями и ролями (Административная панель)"

Данный модуль предоставляет Администратору Платформы централизованные инструменты для управления всеми пользователями и их правами доступа.

- **Управление учетными записями:**

- Реализовать интерфейс для просмотра списка всех зарегистрированных пользователей с возможностью поиска и фильтрации.

- Администратору должны быть доступны следующие действия над учетной записью пользователя: создание (вручную), редактирование профиля, временная блокировка, перманентное удаление.

- **Управление ролями и правами:**

- Реализовать функцию назначения и изменения ролей пользователей (Студент, Преподаватель, Зав. кафедрой, Партнер, Администратор).

- Система должна содержать редактор ролей, позволяющий Администратору гибко настраивать набор разрешений (permissions) для каждой роли (например, разрешение на "создание проекта", "утверждение заявки", "просмотр отчетов").

- **Аудит действий:**

- Система должна вести лог (журнал) всех действий пользователей с высокими привилегиями (Администратор, Зав. кафедрой). В лог должны фиксироваться: выполненное действие, объект действия, время и IP-адрес. Администратор должен иметь доступ к просмотру и фильтрации этого лога.

4.1.9. Модуль "Управление шаблонами документов"

Модуль предназначен для автоматизации документооборота путем создания и использования стандартизированных шаблонов.

- **Функции для Администратора:**

- Реализовать интерфейс для создания, редактирования и удаления шаблонов документов. Система должна поддерживать загрузку шаблонов в форматах .docx и, опционально, .pdf.

- В редакторе шаблонов должна быть реализована система "плейсхолдеров" – специальных меток (например, {{student.full_name}}, {{project.title}}, {{event.date}}), которые при генерации документа будут автоматически заменяться данными из системы. Администратору должен быть доступен список всех возможных плейсхолдеров.

- **Автоматическая генерация документов:**

- Система должна использовать эти шаблоны для автоматического создания документов в различных частях Платформы.
- **Примеры генерируемых документов:**
 - Сертификат участника хакатона.
 - Диплом победителя конкурса.
 - Отчет о выполнении этапа проекта.
 - Заявка на грант.
 - Справка об участии в проекте.
 - Служебная записка о формировании команды.
- **Инициация генерации:**
 - Должна быть предусмотрена возможность инициации создания документа по шаблону уполномоченными пользователями (например, Преподаватель может сгенерировать справку для студента своей группы).

Раздел 5. Требования к архитектуре и технологическому стеку

Настоящий раздел определяет требования к системной архитектуре, технологическому стеку, среде разработки и развертывания "Единой Цифровой Образовательной Платформы Университета" (ЕЦОПУ). Целью данных требований является обеспечение масштабируемости, надежности, безопасности и долгосрочной поддерживаемости Платформы.

5.1. Требования к архитектуре системы

Архитектура Платформы должна быть спроектирована с учетом долгосрочного развития, высоких нагрузок и необходимости независимого обновления функциональных модулей.

• 5.1.1. Архитектурный паттерн: Микросервисная архитектура

Платформа должна быть реализована в виде набора независимых, слабосвязанных сервисов (микросервисов). Каждый сервис должен соответствовать одному или нескольким функциональным модулям, описанным в разделе 4 (например, "Сервис аутентификации", "Сервис проектов", "Сервис уведомлений").

○ Принципы микросервисной архитектуры, которые должны соблюдаться:

- **Единая ответственность (Single Responsibility):** Каждый сервис решает одну четко определенную бизнес-задачу.

- **Автономность:** Каждый сервис может разрабатываться, тестироваться, развертываться и масштабироваться независимо от других. Сбой одного сервиса не должен приводить к полному отказу всей системы (требуется реализация механизмов отказоустойчивости).

- **Собственное хранилище данных:** Каждый сервис должен владеть своей базой данных или схемой данных, чтобы избежать прямых зависимостей на уровне БД. Взаимодействие между сервисами по вопросам данных должно осуществляться исключительно через API.

• 5.1.2. Взаимодействие между сервисами

- **Синхронное взаимодействие:** Для запросов, требующих немедленного ответа (например, получение профиля пользователя при

открытии страницы), должно использоваться синхронное взаимодействие по протоколу HTTP/HTTPS через REST API или gRPC.

- **Асинхронное взаимодействие:** Для операций, не требующих мгновенного ответа и для обеспечения отказоустойчивости (например, отправка email-уведомления после создания проекта, обновление рейтинга после закрытия задачи), должно использоваться асинхронное взаимодействие через брокер сообщений (например, RabbitMQ, Apache Kafka). Это позволяет гарантировать доставку сообщений и снизить связанность сервисов.

- **5.1.3. Шлюз API (API Gateway)**

Весь внешний трафик от клиентских приложений (веб-фронтенд) должен проходить через единую точку входа — Шлюз API.

- **Задачи Шлюза API:**

- Маршрутизация запросов к соответствующим микросервисам.
- Аутентификация и авторизация (проверка токенов доступа).
- Агрегация данных из нескольких сервисов в один ответ.
- Ограничение частоты запросов (Rate Limiting) для защиты от DoS-атак.
- Централизованное логирование запросов.

- **5.1.4. Обнаружение сервисов (Service Discovery)**

В системе должен быть реализован механизм автоматического обнаружения сервисов (например, с использованием Consul, etcd или встроенных средств Kubernetes), чтобы сервисы могли находить друг друга в динамически изменяющейся среде OpenStack.

- **5.1.5. Контейнеризация и оркестрация**

Все компоненты Платформы (микросервисы, базы данных, брокеры сообщений) должны быть упакованы в **Docker-контейнеры**. Это обеспечивает изоляцию, переносимость между средами (разработка, тестирование, продакшн) и консистентность окружения.

Для управления жизненным циклом контейнеров (развертывание, масштабирование, мониторинг, самовосстановление) должен использоваться оркестратор (например **Kubernetes**).

5.2. Требования к технологическому стеку

Выбор технологического стека должен обеспечивать высокую производительность, безопасность, широкую поддержку сообществом и наличие квалифицированных специалистов на рынке.

- **5.2.1. Backend (Серверная часть)**

Рекомендуемый язык/фреймворк: **Python 3.9+** с использованием фреймворка **FastAPI** (для высокопроизводительных сервисов) или **Django/DRF** (для сервисов с комплексной бизнес-логикой и админ-панелью).

Альтернатива: *Java 17+ / Spring Boot, Go.*

- **5.2.2. Frontend (Клиентская часть)**

Рекомендуемый фреймворк: **Vue.js 3** (с использованием Composition API и Pinia для управления состоянием) или **React 18+** (с использованием Hooks и Redux/MobX).

Сборка проекта: Vite или Webpack.

UI-библиотека: Должен использоваться готовый UI-kit (например, Vuetify для Vue, Material-UI для React) или разработан собственный в рамках проекта для обеспечения единообразия.

- **5.2.3. Базы данных и хранилища**

Основная реляционная СУБД: **PostgreSQL 14+**. Должна использоваться для хранения структурированных данных с необходимостью обеспечения транзакционной целостности (данные пользователей, проектов, задач).

In-memory хранилище (кэш): **Redis**. Должен использоваться для кэширования часто запрашиваемых данных (сессии пользователей, результаты тяжелых запросов) для снижения нагрузки на основные СУБД.

Полнотекстовый поиск: **OpenSearch**. Должен использоваться для реализации функции глобального поиска по Платформе.

Хранение файлов: Пользовательские файлы (документы, изображения, архивы) должны храниться в объектном хранилище, совместимом с S3 API (например, **OpenStack Swift** или **Ceph**). Это обеспечивает масштабируемость и отказоустойчивость хранения.

- **5.2.4. Брокер сообщений**

Рекомендуемый брокер: RabbitMQ. Хорошо подходит для сложных сценариев маршрутизации и гарантированной доставки сообщений между микросервисами. *Альтернатива: Apache Kafka для сценариев с потоковой обработкой данных (например, сбор аналитики)*

- **5.2.5 Выбор стека**

Выбор конкретного стека, фреймворка и тд остается за командой разработки, но должен быть единым для всей Платформы.

5.3. Требования к среде развертывания (Deployment)

- **5.3.1. Инфраструктура:** Платформа должна быть развернута на кластере виртуальных машин под управлением облачной платформы **OpenStack**. Должно быть предусмотрено минимум три среды:

- **Development (Среда разработки):** Для работы разработчиков.
- **Staging (Тестовая среда):** Полный клон продакшн-среды для проведения приемочного, регрессионного и нагрузочного тестирования.
- **Production (Рабочая среда):** Среда, доступная конечным пользователям.

- **5.3.2. CI/CD (Непрерывная интеграция и доставка)**

Должен быть настроен CI/CD-пайплайн с использованием **GitLab CI/CD** или **Jenkins**.

Процесс CI: Каждый push в репозиторий должен запускать автоматическую сборку Docker-образов, запуск статических анализаторов кода (linting) и выполнение модульных/интеграционных тестов.

Процесс CD: При успешном прохождении CI и слиянии в основную ветку должна происходить автоматическая доставка (развертывание) обновленных сервисов сначала на Staging-среду, а после ручного

подтверждения (или успешного прохождения автотестов) — на Production-среду с использованием стратегии Blue-Green или Canary для минимизации простоев.

- **5.3.3. Мониторинг и логирование**

Мониторинг: Должна быть развернута система мониторинга на основе стека **Prometheus + Grafana**. Prometheus должен собирать метрики со всех компонентов системы (микросервисы, БД, брокеры, Kubernetes). Grafana должна использоваться для создания дашбордов для визуализации состояния системы в реальном времени.

Логирование: Должна быть внедрена система централизованного сбора логов на основе стека **EFK (Elasticsearch, Fluentd, Kibana)** или **PLG (Promtail, Loki, Grafana)**. Fluentd/Promtail должен агрегировать логи из всех контейнеров, Elasticsearch/Loki — индексировать их для поиска, а Kibana/Grafana — предоставлять интерфейс для анализа и поиска по логам.

Трассировка: Для отладки запросов в микросервисной архитектуре должна быть внедрена система распределенной трассировки, например, **Jaeger** или **Zipkin**.

- **5.2.5 Выбор среды развертывания**

Выбор среды развертывания и тд остается за командой разработки, но должен быть единым для всей Платформы.

Раздел 6. Требования к интеграциям (низкий приоритет)

Настоящий раздел определяет требования к взаимодействию "Единой Цифровой Образовательной Платформы Университета" (ЕЦОПУ) с внешними и внутренними информационными системами. Для каждой интеграции должны быть обеспечены безопасность, надежность и документированность API.

6.1. Интеграция с системами Университета (СУБД / ERP / LMS)

Эта интеграция является важной для автоматического наполнения Платформы базовыми данными и обогащения профилей пользователей.

• 6.1.1. Цели интеграции:

- **Импорт структуры Университета:** Получение и периодическая синхронизация данных о факультетах, кафедрах, учебных группах.
- **Импорт и синхронизация пользователей:** Автоматическое создание и обновление профилей студентов и преподавателей на основе данных из кадровой/студенческой системы.
- **Импорт данных об успеваемости (из LMS):** Получение оценок студентов по дисциплинам для использования в расчете рейтинга.
- **Анализ компетенций:** Получение данных об учебных планах и образовательных программах для сопоставления с навыками, развиваемыми в проектах.

• 6.1.2. Способ интеграции:

- **Предпочтительный метод:** Взаимодействие через **REST API**, предоставляемый системами Университета. ЕЦОПУ выступает в роли клиента.
- **Альтернативный метод (при отсутствии API):** Регулярная (ежесуточная) пакетная выгрузка данных из систем Университета в промежуточные файлы (формат .csv или .xml) на защищенный SFTP-сервер. ЕЦОПУ будет забирать и обрабатывать эти файлы.
- **Требования к API со стороны ЕЦОПУ:**
 - API должен быть документирован в формате **OpenAPI 3.0 (Swagger)**.

- Доступ к API должен быть защищен (например, с помощью **OAuth 2.0** или статических API-ключей).

- Должны быть реализованы методы для получения как полных списков (например, всех студентов), так и для получения изменений с последней синхронизации (delta-обновления).

- **6.1.3. Сценарии и форматы данных:**

Синхронизация структуры и пользователей (частота: раз в 24 часа, в ночное время):

- ЕЦОПУ запрашивает список факультетов и кафедр.
- ЕЦОПУ запрашивает список студентов и преподавателей. Для каждого пользователя должны передаваться: уникальный_ID_в_системе_вуза, ФИО, корпоративный_email, статус (активен/отчислен/уволен), роль (студент/преподаватель), ID_кафедры/ID_группы.

- При обработке ЕЦОПУ создает новых пользователей, деактивирует отсутствующих в выгрузке и обновляет данные у существующих.

- **Получение данных об успеваемости (частота: раз в 24 часа или по запросу):**

- ЕЦОПУ запрашивает итоговые оценки по дисциплинам за закрытые семестры для конкретного студента по его уникальному_ID.

- Формат ответа: массив объектов, где каждый объект: ID_дисциплины, название_дисциплины, итоговая_оценка, дата_сдачи.

- **6.1.4. Обработка ошибок:**

- Должен быть реализован механизм логирования всех операций интеграции.

- В случае недоступности API или ошибок в файлах выгрузки, ЕЦОПУ должна предпринять несколько повторных попыток подключения с возрастающим интервалом.

- При невозможности провести синхронизацию, система должна отправить уведомление Администратору Платформы с деталями ошибки.

6.2. Интеграция с сервисом единого входа (SSO)

- **6.2.1. Цель интеграции:** Обеспечить бесшовную и безопасную аутентификацию пользователей через их единую университетскую учетную запись.
- **6.2.2. Способ интеграции:**
 - **Протоколы:** Система должна поддерживать стандартные протоколы для SSO, такие как **SAML 2.0** или **OpenID Connect (OIDC)**, в зависимости от того, что реализовано на стороне Identity Provider'a Университета (например, ADFS, Keycloak).
 - **Сценарий взаимодействия (SAML):**
 1. Пользователь нажимает "Войти через SSO" на странице входа ЕЦОПУ.
 2. ЕЦОПУ (Service Provider) перенаправляет пользователя на страницу входа Identity Provider'a Университета с SAML-запросом.
 3. Пользователь вводит свои университетские логин и пароль.
 4. Identity Provider аутентифицирует пользователя и перенаправляет его обратно на специальный URL в ЕЦОПУ (Assertion Consumer Service URL) с SAML-ответом, содержащим атрибуты пользователя (уникальный_ID, email, ФИО).
 5. ЕЦОПУ проверяет цифровую подпись SAML-ответа, находит пользователя по уникальному_ID и авторизует его в системе.

6.3. Интеграция с Telegram Bot (Пример "EventVoyager")

- **6.3.1. Цели интеграции:**
 - Оперативное информирование пользователей о событиях на Платформе.
 - Предоставление простых функций взаимодействия с Платформой через популярный мессенджер.
- **6.3.2. Способ интеграции:** Взаимодействие через **Telegram Bot API**. ЕЦОПУ выступает в роли сервера, который обрабатывает запросы от Telegram (webhook).

- **6.3.3. Функциональные требования к боту:**

- **Привязка аккаунта:** Пользователь в своем профиле на Платформе должен иметь возможность привязать свой Telegram-аккаунт, перейдя по уникальной ссылке в бота.

- **Рассылка уведомлений (односторонняя связь):**
 - Бот должен рассылать персонализированные уведомления о новых задачах, приглашениях в проект, приближающихся дедлайнах.

- Пользователь должен иметь возможность подписаться на тематические каналы (например, "Хакатоны по ИИ", "Конференции по экономике"), куда бот будет рассылать анонсы новых мероприятий.

- **Двусторонняя связь (интерактивные команды):**
 - /start – приветственное сообщение и основное меню.
 - /my_tasks – получение списка 5 ближайших по дедлайну задач.
 - /events – получение списка 3 ближайших мероприятий с кнопками для быстрой регистрации.

- Реализация **Inline-кнопок** под сообщениями (например, "Зарегистрироваться на мероприятие", "Открыть на платформе").

- **(Опционально) Парсинг внешних источников:** Бот может быть наделен функцией периодического парсинга заданных внешних сайтов (например, анонсы-конференций.рф) и отправки найденных релевантных мероприятий в общие каналы.

6.4. Интеграция с сервисами email-рассылок

- **6.4.1. Цель интеграции:** Обеспечить надежную доставку транзакционных писем (подтверждение регистрации, восстановление пароля) и массовых рассылок (дайджесты, анонсы).

- **6.4.2. Способ интеграции:** Взаимодействие через **SMTP API** или **HTTP API** внешнего почтового сервиса (например, SendGrid, Postmark, или собственный почтовый релей Университета).

- **6.4.3. Требования:**

- Все письма должны отправляться с использованием **HTML-шаблонов**, брендированных в стиле Университета.

- Система должна отслеживать статусы доставки писем (доставлено, открыто, клик, ошибка) через webhook'и, предоставляемые сервисом рассылок, для анализа и диагностики проблем.

6.5. Интеграция с внешними календарями

- **6.5.1. Цель интеграции:** Позволить пользователям легко добавлять события Платформы (дедлайны проектов, даты мероприятий) в свои личные календари.

- **6.5.2. Способ интеграции:**

- **Экспорт одиночных событий:** На странице каждого мероприятия или задачи с дедлайном должна быть кнопка "Добавить в календарь", которая генерирует и предлагает скачать .ics файл (стандарт iCalendar).

- **Подписка на календарь:** В профиле пользователя должна быть уникальная, защищенная ссылка на его персональный календарь ЕЦОПУ в формате iCalendar. Пользователь сможет добавить эту ссылку в свой Google Calendar, Outlook, Apple Calendar и др. для автоматической синхронизации всех своих событий с Платформы.

6.6. Интеграция с системой "Мегалаборатории" (или иными системами бронирования ресурсов)

- **6.6.1. Цель интеграции:** Обеспечить командам проектов возможность просматривать доступность и бронировать необходимое оборудование или помещения (например, 3D-принтер, переговорная комната, рабочее место в коворкинге).

- **6.6.2. Способ интеграции:** Взаимодействие через **REST API**, предоставляемый системой "Мегалаборатории".

- **6.6.3. Сценарии взаимодействия:**

- **Просмотр доступности:** На странице проекта в ЕЦОПУ должна быть возможность запросить список доступного оборудования/помещений с отображением их календаря занятости (свободные слоты).
- **Бронирование ресурса:** Руководитель проекта должен иметь возможность отправить запрос на бронирование ресурса на определенное время. Запрос должен содержать ID_проекта, ID_пользователя, ID_ресурса, время_начала, время_окончания.
- **Статус бронирования:** ЕЦОПУ должна периодически опрашивать статус заявки на бронирование (одобрена, отклонена) и уведомлять руководителя

Раздел 7. Этапы, порядок разработки и приемки работ

Настоящий раздел определяет последовательность этапов разработки, методологию управления проектом, а также процедуры контроля и приемки выполненных работ по созданию "Единой Цифровой Образовательной Платформы Университета" (ЕЦОПУ).

7.1. Этапы выполнения работ

Проект делится на несколько крупных последовательных этапов, каждый из которых имеет свои цели и критерии завершения.

- **7.1.1. Этап 0: Подготовительный (Срок: 2-3 недели)**
 - **Цели:** Финализация требований, формирование команды, настройка инфраструктуры.
 - **Состав работ:**
 1. Формирование и утверждение совместной рабочей группы (Заказчик, Исполнитель).
 2. Проведение серии установочных встреч и воркшопов для детального обсуждения и финализации данного ТЗ.
 3. Разработка и согласование **Программы и методики испытаний (ПМИ)**.
 4. Настройка сред разработки, тестирования (Staging) и CI/CD пайплайна.
 5. Настройка системы управления проектами, первоначальное наполнение бэклога продукта.
 - **Результат этапа:** Утвержденное ТЗ, утвержденная ПМИ, настроенная рабочая среда, готовый к работе бэклог.
- **7.1.2. Этап 1: Проектирование и прототипирование (Срок: 4-6 недель)**
 - **Цели:** Визуализация будущего продукта, проверка ключевых гипотез юзабилити, финализация архитектуры.
 - **Состав работ:**

1. Разработка детальных схематичных прототипов (**wireframes**) для ключевых пользовательских сценариев (регистрация, создание проекта, поиск мероприятия, просмотр портфолио).

2. Создание **кликабельного интерактивного прототипа** на основе wireframes (с использованием инструментов Figma, Axure и т.п.).

3. Проведение **юзабилити-тестирования** прототипа на фокус-группе из реальных пользователей (студенты, преподаватели). Сбор и анализ обратной связи.

4. Разработка и утверждение **UI Kit / Design System** (набор компонентов интерфейса, цветовая палитра, типографика).

5. Детальное проектирование и документирование архитектуры системы (схемы микросервисов, форматы API, структура БД).

◦ **Результат этапа:** Утвержденный и протестированный интерактивный прототип, утвержденный UI Kit, детальное описание архитектуры (Technical Design Document).

• **7.1.3. Этап 2: Разработка Минимально жизнеспособного продукта (MVP) (Срок: 1-3 месяцев)**

◦ **Цели:** Создание первой работающей версии Платформы с базовым, но критически важным функционалом для запуска и получения реальной обратной связи.

◦ **Состав работ:**

1. Разработка функционала в соответствии с определенным скоупом MVP (например, Модули 0, 1, 2, 4 из Раздела 4.1).

2. Регулярные демонстрации результатов в конце каждого двухнедельного спринта.

3. Проведение функционального, интеграционного и регрессионного тестирования после каждого спринта.

4. Подготовка пользовательской и технической документации для версии MVP.

5. Проведение нагрузочного тестирования на Staging-среде.

- **Результат этапа:** Развернутая на Production-среде, протестированная и готовая к пилотному запуску версия MVP Платформы. Полный комплект документации для MVP.
- **7.1.4. Этап 3: Пилотное внедрение и опытная эксплуатация (Срок: 2-3 месяца)**
 - **Цели:** Проверка работы Платформы в реальных условиях на ограниченной группе пользователей, сбор обратной связи и выявление потребностей для дальнейшего развития.
 - **Состав работ:**
 1. Запуск MVP в опытную эксплуатацию на 1-2 пилотных факультетах/кафедрах.
 2. Обучение пользователей пилотной группы.
 3. Осуществление технической поддержки пользователей пилотной группы.
 4. Активный сбор обратной связи через опросы, интервью, встроенные формы.
 5. Исправление дефектов, выявленных в ходе опытной эксплуатации.
 6. Формирование и приоритизация бэклога для следующего этапа на основе полученной обратной связи.
 - **Результат этапа:** Отчет о результатах опытной эксплуатации, включающий статистику использования, собранную обратную связь и список выявленных дефектов. Приоритизированный бэклог для дальнейшего развития.
- **7.1.5. Этап 4: Развитие функционала и полномасштабное внедрение (постоянный процесс)**
 - **Цели:** Поэтапное наращивание функциональности Платформы и масштабирование на весь Университет.
 - **Состав работ:**

1. Разработка и внедрение модулей, не вошедших в MVP (например, продвинутая аналитика, рейтинговая система, портал для партнеров).
 2. Доработка существующего функционала на основе бэклога, сформированного на предыдущем этапе.
 3. Работа продолжается в режиме двухнедельных спринтов с регулярными релизами нового функционала.
 4. Постепенное подключение к Платформе остальных факультетов и кафедр Университета.
- **Результат этапа:** Регулярно обновляемая и развивающаяся Платформа, полностью внедренная в процессы Университета.

Раздел 8. Требования к документированию

Настоящий раздел определяет состав, содержание, формат и порядок предоставления комплекта документации, разрабатываемой и поставляемой в ходе создания и внедрения "Единой Цифровой Образовательной Платформы Университета" (ЕЦОПУ). Вся документация должна быть выполнена на русском языке и поддерживаться в актуальном состоянии на протяжении всего жизненного цикла проекта.

8.1. Состав комплекта документации

Комплект документации должен включать в себя три основных типа документов: проектную, пользовательскую и техническую (эксплуатационную).

8.2. Проектная документация

Эта документация фиксирует ключевые решения и договоренности на всех этапах проекта.

• 8.2.1. Техническое Задание (ТЗ)

Настоящий документ, финализированный и утвержденный обеими сторонами на подготовительном этапе. Любые последующие изменения в требованиях должны оформляться через **Протоколы управления изменениями** и вноситься в новые версии ТЗ.

- **Формат:** Электронный документ в формате .docx и .pdf.

• 8.2.2. Программа и методика испытаний (ПМИ)

- **Содержание:** Документ, описывающий объем, порядок, методы и условия проведения приемочных испытаний. Должен включать:
 - Объект испытаний (версия ПО, конфигурация тестовой среды).
 - Цели испытаний.
 - Перечень проверяемых требований (ссылки на пункты ТЗ).
 - **Тест-кейсы:** Детальные пошаговые сценарии для проверки каждой функции с указанием входных данных, ожидаемых результатов и критериев успешности прохождения.

- Методы проведения нагрузочного тестирования и критерии его успешности.
- Состав испытательной комиссии.
- **Формат:** Электронный документ в формате .docx и .pdf.
- **8.2.3. Протоколы испытаний и Акты сдачи-приемки работ**
- **Содержание:** Документы, формально фиксирующие результаты проведения испытаний и факт выполнения работ по каждому этапу.
- **Формат:** Электронные сканы подписанных документов в формате .pdf.

8.3. Пользовательская документация

Предназначена для конечных пользователей Платформы и должна быть написана простым, понятным языком, без использования технического жаргона.

- **8.3.1. Руководство пользователя**
 - **Структура:** Должно быть разработано отдельное руководство для каждой ключевой роли: **Студент, Преподаватель, Заведующий кафедрой, Внешний партнер.**
 - **Содержание для каждой роли:**
 1. **Введение:** Краткое описание возможностей Платформы для данной роли.
 2. **Начало работы:** Пошаговые инструкции по регистрации, первому входу, настройке профиля и знакомству с интерфейсом (дашбордом).
 3. **Основные сценарии работы:** Детальное пошаговое описание выполнения всех ключевых задач, доступных для данной роли (например, для Студента: "Как найти проект и подать заявку", "Как создать команду", "Как сдать задачу", "Как пополнить свое портфолио"). Каждое действие должно сопровождаться **актуальными скриншотами интерфейса** с поясняющими выносками.
 4. **Справочный раздел:** Описание всех элементов интерфейса (кнопок, иконок, разделов меню).

5. **Часто задаваемые вопросы (FAQ):** Ответы на наиболее вероятные вопросы пользователей.

○ **Формат:**

- **Онлайн-справка (Help Center / База знаний):** Предпочтительный формат. Должна быть интегрирована в Платформу, с возможностью контекстного вызова (например, кнопка "?" на странице создания проекта открывает соответствующий раздел справки) и полнотекстовым поиском по всем статьям.
- **PDF-версии:** Возможность скачать полное руководство для каждой роли в формате .pdf для офлайн-просмотра.

8.4. Техническая (эксплуатационная) документация

Предназначена для IT-специалистов, которые будут сопровождать, поддерживать и развивать Платформу.

• **8.4.1. Описание архитектуры (Technical Design Document)**

○ **Содержание:**

1. **Обзор архитектуры:** Высокоуровневое описание архитектурного паттерна (микросервисы), ключевых компонентов и их взаимодействия.

2. **Диаграммы:**

- **Контекстная диаграмма (C4 Model - Level 1):**

Система и ее взаимодействие с пользователями и внешними системами.

- **Контейнерная диаграмма (C4 Model - Level 2):**

Ключевые контейнеры (веб-фронтенд, API Gateway, микросервисы, БД) и связи между ними.

- **Диаграмма развертывания (Deployment Diagram):**

Схема размещения компонентов системы на инфраструктуре OpenStack и Kubernetes.

3. **Описание микросервисов:** Для каждого микросервиса должно быть указано его назначение, используемый технологический стек, предоставляемый API, зависимости от других сервисов.

4. **Описание модели данных:** Логическая и физическая схема базы данных (ER-диаграммы) для ключевых СУБД (PostgreSQL).

5. **Описание принятых архитектурных решений:** Обоснование выбора ключевых технологий и паттернов (например, "Почему для асинхронного взаимодействия был выбран RabbitMQ, а не Kafka").

- **8.4.2. Документация на API (API Reference)**

Содержание: Полное описание всех публичных API-эндпоинтов, предоставляемых микросервисами и API Gateway.

Формат: Документация должна генерироваться автоматически на основе кода и комментариев с использованием стандарта **OpenAPI 3.0 (Swagger)**. Должен быть развернут интерактивный **Swagger UI**, позволяющий разработчикам не только читать описание методов, но и выполнять тестовые запросы прямо из браузера.

- **Описание каждого эндпоинта должно включать:**

- HTTP-метод (GET, POST, PUT, DELETE).
- URL-путь.
- Краткое описание назначения.
- Параметры запроса (в пути, в query-строке, в заголовках, в теле).
- Структура тела запроса (JSON Schema).
- Возможные коды и структуры ответов (успешные и ошибочные).
- Требования к аутентификации/авторизации.
- Примеры запросов и ответов.

- **8.4.3. Руководство по развертыванию и эксплуатации (DevOps Guide)**

- **Содержание:**

0. **Требования к окружению:** Описание требований к инфраструктуре OpenStack (версии компонентов, конфигурация сети, хранилища).

1. **Инструкция по развертыванию:** Пошаговое руководство по развертыванию всей системы "с нуля" с использованием предоставленных скриптов (IaC - Terraform/Ansible, Helm-чарты для Kubernetes).

2. **Инструкция по обновлению:** Описание процедуры обновления компонентов системы до новой версии.

3. **Инструкция по резервному копированию и восстановлению:** Детальное описание процедур и скриптов для создания резервных копий и восстановления системы после сбоя.

4. **Руководство по мониторингу:** Описание ключевых метрик системы, которые необходимо отслеживать (в Prometheus), и как интерпретировать дашборды в Grafana.

5. **Карта типовых проблем и способы их решения (Troubleshooting Guide):** Описание наиболее вероятных сбоев и пошаговые инструкции по их диагностике и устранению.

- **8.4.4. Руководство разработчика (Developer's Guide)**

- **Содержание:**

0. **Инструкция по настройке локального окружения:** Как разработчику развернуть систему на своей машине для разработки и отладки (с использованием Docker Compose).

1. **Соглашения по кодированию (Coding Conventions):** Правила оформления кода, именования переменных, структура проектов для каждого языка/фреймворка.

2. **Процесс работы с системой контроля версий (Git Flow):** Описание правил ветвления, создания merge-реквестов, проведения code review.

3. **Инструкция по сборке и запуску тестов.**

8.5. Порядок предоставления и актуализации

- Первичная версия всей документации должна быть передана Заказчику вместе со сдачей MVP.

- Документация должна обновляться и поставляться с каждым последующим крупным релизом Платформы.

- Вся документация должна храниться в электронном виде в согласованной системе (например, Confluence для текстовых документов, GitLab/GitHub Wiki для технической документации) с контролем версий.

Раздел 9. Порядок приемки и испытаний

Настоящий раздел определяет процедуры, критерии и порядок проведения испытаний и приемки "Единой Цифровой Образовательной Платформы Университета" (ЕЦОПУ) с целью проверки соответствия разработанного продукта требованиям настоящего Технического Задания (ТЗ).

9.1. Общие положения

• 9.1.1. Цель испытаний:

- Удостовериться, что весь реализованный функционал Платформы работает корректно и в полном соответствии с функциональными требованиями.
- Подтвердить, что Платформа соответствует заявленным нефункциональным требованиям к производительности, надежности, безопасности и юзабилити.
- Проверить полноту и качество разработанной пользовательской и технической документации.
- Принять официальное решение о готовности Платформы (или ее очередного инкремента/версии) к вводу в опытную или промышленную эксплуатацию.

• 9.1.2. Объект испытаний:

- Программное обеспечение Платформы ЕЦОПУ определенной версии, развернутое на тестовой среде (Staging), конфигурация которой идентична или максимально приближена к производственной (Production).
- Комплект сопроводительной документации (проектной, пользовательской, технической).

• 9.1.3. Виды испытаний:

- **Предварительные (внутренние) испытания:** Проводятся силами Исполнителя перед передачей системы Заказчику для проверки готовности к приемочным испытаниям.
- **Приемочные испытания:** Проводятся совместной комиссией для принятия решения о приемке работ.

9.2. Программа и методика испытаний

• **9.2.1. Разработка и согласование:**

- Исполнитель разрабатывает детальный документ "Программа и методика испытаний" на основе настоящего ТЗ.

• **9.2.2. Содержание ПМИ:**

- **Раздел 1: Общие положения.** (Цели, объект испытаний, состав комиссии).
- **Раздел 2: Перечень проверяемых требований.** Таблица, сопоставляющая каждый пункт ТЗ с соответствующими тест-кейсами в ПМИ.
- **Раздел 3: Условия и порядок проведения испытаний.** (Место проведения, необходимое ПО и оборудование, продолжительность, роли и обязанности участников).
- **Раздел 4: Методика функционального тестирования.**
 - **Тест-кейсы:** Для каждого функционального требования из Раздела 4.1 должен быть описан как минимум один тест-кейс. Каждый тест-кейс должен содержать:
 - ID Тест-кейса
 - Название (например, "Создание нового проекта преподавателем")
 - Ссылка на пункт ТЗ
 - Предусловия (например, "Пользователь авторизован с ролью 'Преподаватель'")

- Шаги для воспроизведения (детальное описание действий пользователя)
- Ожидаемый результат (четкое описание того, что должно произойти)
- Статус (пройден/не пройден/заблокирован)
- Комментарий (для фиксации дефектов)

○ **Раздел 5: Методика нефункционального тестирования.**

- **Нагрузочное тестирование:** Описание сценариев и инструментов (например, JMeter, K6) для имитации нагрузки (5000 одновременных пользователей). Критерии успешности: среднее время отклика, процент ошибок не более 0.1%, утилизация CPU/RAM не более 80%.
- **Тестирование безопасности:** Описание методики проверки на уязвимости из списка OWASP Top 10 (может включать использование автоматических сканеров и ручную проверку).
- **Тестирование юзабилити:** Проверка на соответствие требованиям UI/UX, включая адаптивность на разных устройствах и кроссбраузерность.

○ **Раздел 6: Критерии приемки.** Четкое определение условий, при которых система считается прошедшей испытания.

○ **Раздел 7: Формы отчетных документов.** Шаблоны Протокола испытаний и Ведомости дефектов.